

# FENIXCMS\_2

## FASE 2 - CHECKOUTSESSION REAL, PAYPAL APPROVAL/CAPTURE, PROVISIONING ATÓMICO Y ELIMINACIÓN DE CREDENCIALES INSEGURAS

Documento técnico de implementación para AI Studio.

Repositorio objetivo: fenixcmssl-spec/moreno

Rama: main

Prioridad: CRÍTICA

### INICIO FASE FENIXCMS\_2

---

## 1. FINALIDAD DE ESTA FASE

FENIXCMS\_1 corrigió el antiguo endpoint de provisioning gratuito y añadió la integración server-side con PayPal. La auditoría posterior de main demuestra que todavía hay que completar el ciclo comercial real.

La Fase 2 debe convertir el flujo actual en:

Cliente selecciona Application + Plan  
-> servidor valida catálogo PostgreSQL  
-> servidor crea CheckoutSession  
-> servidor crea PayPal Order real  
-> servidor devuelve approval URL real  
-> navegador redirige al comprador a PayPal  
-> comprador aprueba  
-> PayPal devuelve al comprador a FenixCMS  
-> FenixCMS captura server-side  
-> FenixCMS valida captura  
-> provisioning atómico  
-> CheckoutSession COMPLETED  
-> acceso del cliente al CMS.

REGLA ABSOLUTA:

NO PAYMENT CAPTURED = NO PROVISIONING

El retorno del navegador desde PayPal no constituye por sí mismo una confirmación de pago.

---

## 2. PROBLEMAS QUE DEBE CORREGIR FENIXCMS\_2

La revisión del código actual identifica como mínimo:

1. SaasLanding llama al capture inmediatamente después del checkout en lugar de redirigir al comprador a PayPal.
2. No existe una CheckoutSession persistente independiente del Tenant.
3. Payment exige tenantId aunque el Tenant final debe crearse después del pago.
4. El flujo superior crea Tenant/License en una transacción y después crea Subscription e Invoice en operaciones separadas.
5. Algunos errores críticos son capturados y convertidos en warning.
6. auth.service.ts conserva usuarios/contraseñas maestras hardcodedas y fallback en memoria.
7. WebhookEvent dispone de fallback en memoria y no debe ser fuente de verdad en producción.
8. Existen fallbacks INITIAL\_\* en servicios administrativos.
9. Stripe todavía dispone de lógica de URL que no representa una Stripe Checkout Session real.
10. Los tests actuales diferencian insuficientemente entre mock, sandbox y producción.

FENIXCMS\_2 no debe intentar resolver en esta fase todo el almacenamiento real, SEO, editor completo, DNS/SSL avanzado ni rate limiting distribuido. Esos elementos quedan para fases posteriores.

---

## 3. ARQUITECTURA FINAL

La arquitectura debe separar claramente las responsabilidades:

ROUTE

-> VALIDATOR

-> CHECKOUT SERVICE

-> PAYPAL GATEWAY

-> CHECKOUT SESSION  
-> PROVISIONING SERVICE  
-> PRISMA/POSTGRESQL

El navegador nunca debe ser propietario de:

- importe final;
- moneda final;
- Tenant final;
- License;
- Subscription;
- Invoice;
- entitlements;
- estado de pago.

Fuente de autoridad:

1. PayPal para el estado externo del pago.
2. PostgreSQL para CheckoutSession y recursos internos.
3. Browser únicamente como iniciador de acciones.

---

## 4. FASE 2A - CHECKOUTSESSION

### Objetivo

Crear una entidad persistente de plataforma que represente una intención de compra antes del Tenant.

Debe ser independiente del contexto tenant-scoped.

### Datos mínimos

id  
provider  
status  
applicationId  
planId  
billingPeriod  
amountExpected  
currencyExpected  
customerName  
customerEmail  
tenantName  
tenantSlug  
billingAddress  
paypalOrderId  
paypalCaptureId  
paymentId  
tenantId  
expiresAt  
approvedAt  
capturedAt  
completedAt  
consumedAt  
failureCode  
failureReason  
createdAt  
updatedAt

### Estados recomendados

CREATED  
PENDING\_APPROVAL  
APPROVED  
CAPTURE\_PENDING  
PROVISIONING  
COMPLETED  
FAILED  
EXPIRED  
CANCELLED

## Reglas de índices y unicidad

paypalOrderId debe ser único cuando exista.

paypalCaptureId debe ser único cuando exista.

paymentId debe ser único cuando exista.

tenantId debe ser único cuando exista en CheckoutSession.

Indexar status.

Indexar expiresAt.

Indexar customerEmail.

Indexar applicationId + planId.

## Regla de seguridad

CheckoutSession no debe pertenecer al conjunto de modelos tenant-scoped de Prisma.

No debe depender de getCurrentTenantId().

---

## 5. FASE 2B - MIGRACIÓN PRISMA/POSTGRESQL

Modificar:

prisma/schema.prisma

Crear una migración nueva.

NO modificar migraciones históricas.

NO ejecutar prisma migrate reset en producción.

## Dinero

El campo amountExpected de CheckoutSession debe utilizar Decimal con precisión de dos decimales.

En las comparaciones críticas utilizar Decimal y no Float.

No es obligatorio convertir todos los importes de FenixCMS en esta fase; la migración completa de Float queda para una fase específica.

---

## 6. FASE 2C - SOURCE OF TRUTH DEL CATÁLOGO

Modificar:

lib/services/plan.service.ts

La comprobación de checkout debe garantizar:

Application existe.

Plan existe.

Plan pertenece a Application.

Plan está activo.

Billing period es válido.

Precio existe.

Moneda existe.

No se debe utilizar:

- primera Application;
- primer Plan;
- INITIAL\_APPLICATIONS;
- INITIAL\_PLANS;
- creación automática de ECOMMERCE si la Application solicitada no existe.

Si la combinación es inválida:

NO CREAR PAYPAL ORDER.

---

## 7. FASE 2D - SNAPSHOT COMERCIAL

En el momento de crear CheckoutSession, guardar un snapshot inmutable:

applicationId  
planId  
planName  
billingPeriod  
amountExpected  
currencyExpected  
customerName  
customerEmail  
tenantName  
tenantSlug  
billingAddress  
expiresAt

Este snapshot representa exactamente lo que el comprador está intentando adquirir.

Si un administrador cambia posteriormente el precio del plan, la sesión existente conserva su precio esperado.

El capture no debe recalcular el importe desde el navegador.

---

## 8. FASE 2E - VALIDACIÓN DEL POST /api/billing/checkout

Modificar:

app/api/billing/checkout/route.ts

### Entrada permitida

applicationId  
planId  
billingPeriod  
customerName  
customerEmail  
tenantName  
tenantSlug  
billingAddress

### No aceptar como autoridad

amount  
currency  
tenantId  
licenseKey  
paymentId  
providerTransactionId  
subscriptionId  
invoiceId  
activationLimit  
entitlements  
status

Se puede rechazar el payload por campos inesperados o ignorarlos, pero nunca utilizarlos como fuente de verdad.

### Validación

Usar Zod o los validators ya existentes.

Validar:

- email;
- longitud de nombre;
- longitud de tenantName;
- longitud de tenantSlug;
- billingPeriod;

- application;
- plan;
- billingAddress;
- tamaño total del JSON.

---

## 9. FASE 2F - VALIDACIÓN DEL TENANT SLUG

El slug debe ser server-side y unique.

Formato recomendado:

[a-z0-9-]

Normalización:

lowercase

trim

eliminar caracteres inválidos

Debe impedir slug reservado.

Reservados mínimos:

admin

api

billing

login

signup

support

www

mail

smtp

webhook

assets

static

dashboard

super-admin

No utilizar Date.now como único mecanismo de unicidad.

PostgreSQL debe mantener la restricción unique.

---

## 10. FASE 2G - CREATE CHECKOUTSESSION

Secuencia obligatoria:

1. Validar request.
2. Resolver Application.
3. Resolver Plan.
4. Validar relación Application -> Plan.
5. Validar estado del Plan.
6. Calcular importe desde PostgreSQL.
7. Crear CheckoutSession.
8. Crear PayPal Order real.
9. Guardar paypalOrderId.
10. Cambiar estado a PENDING\_APPROVAL.
11. Devolver approval URL.

Si PayPal falla:

- no crear Tenant;
- no crear License;
- no crear Subscription;
- no crear Invoice.

La CheckoutSession puede quedar FAILED o en estado recuperable según el fallo.

---

## 11. FASE 2H - PAYPAL CREATE ORDER

Modificar/validar:

lib/services/paypal-gateway.service.ts

La entrada del gateway debe venir exclusivamente del servidor.

Valores:

intent = CAPTURE

reference\_id = referencia estable de CheckoutSession

custom\_id = CheckoutSession ID

currency\_code = currencyExpected

value = amountExpected

La respuesta debe devolver la approval URL entregada por PayPal.

NO construir manualmente una URL de PayPal si PayPal no la ha entregado.

Documentación oficial:

<https://developer.paypal.com/api/rest/integration/orders-api>

---

## 12. FASE 2I - PAYPAL-REQUEST-ID

Cuando corresponda, utilizar PayPal-Request-Id como protección externa contra duplicación.

Además deben existir unique constraints internas de PostgreSQL.

No depender exclusivamente de PayPal-Request-Id.

Documentación:

<https://developer.paypal.com/sdk/orders/v2/orders-create/>

---

## 13. FASE 2J - REDIRECT DEL CLIENTE

Modificar:

components/saas/SaaS Landing.tsx

El flujo correcto es:

submit

-> POST /api/billing/checkout

-> obtener approvalUrl real

-> window.location.assign(approvalUrl)

No ejecutar capture inmediatamente.

La UI debe indicar:

Preparando pago seguro.

Redirigiendo a PayPal.

Nunca debe decir:

Pago completado.

antes de la captura real.

---

## 14. FASE 2K - BILLING SUCCESS

Crear o corregir:

app/billing/success/page.tsx

El retorno de PayPal sirve para iniciar la reconciliación.

NO significa que Payment esté COMPLETED.

La página debe:

1. mostrar Verificando pago;
2. obtener el identificador de la Order;
3. llamar al servidor;
4. permitir al servidor buscar CheckoutSession;
5. ejecutar capture;
6. esperar el resultado;
7. mostrar éxito solo si provisioning está confirmado.

No guardar como evidencia de pago un valor recibido solamente por query string.

---

## 15. FASE 2L - BILLING CANCEL

Crear o corregir:

app/billing/cancel/page.tsx

No debe crear:

Tenant  
License  
Subscription  
Invoice

Si la CheckoutSession está todavía abierta:

PENDING\_APPROVAL -> CANCELLED

No borrar histórico.

---

## 16. FASE 2M - POST /api/billing/capture

Modificar:

app/api/billing/capture/route.ts

Entrada mínima:

paypalOrderId

El servidor debe obtener de CheckoutSession:

- application;
- plan;
- billing period;
- expected amount;
- expected currency;
- customer;
- tenant slug;
- tenant name;
- billing address.

Nunca utilizar un segundo request del browser para sobrescribir esos valores.

---

## 17. FASE 2N - FLUJO DE CAPTURE

Secuencia:

paypalOrderId  
-> buscar CheckoutSession  
-> comprobar existencia  
-> comprobar expiración  
-> comprobar estado  
-> capture PayPal server-side  
-> validar respuesta PayPal  
-> provisioning atómico  
-> marcar COMPLETED  
-> devolver resultado.

Estados:

Si no existe:  
404.

Si expiró:  
410/409.

Si ya está COMPLETED:  
respuesta idempotente.

Si capture no está COMPLETED:  
no provisionar.

Si una comprobación de seguridad falla:  
409.

---

## 18. FASE 20 - VALIDACIÓN EXACTA DE CAPTURE

Nunca aceptar únicamente:

capture status = COMPLETED

Validar:

### Order ID

PayPal Order ID == CheckoutSession.paypalOrderId

### Capture ID

Debe existir y ser real.

### Capture status

COMPLETED

### Amount

captured amount == CheckoutSession.amountExpected

Comparar con Decimal.

### Currency

captured currency == CheckoutSession.currencyExpected

### Reference

custom\_id/reference\_id debe corresponder a la CheckoutSession.

### Reutilización

captureId ya utilizado = no repetir provisioning.

Documentación PayPal:

<https://developer.paypal.com/platforms/checkout/standard/integrate>

---

## 19. FASE 2P - SAAS PROVISIONING SERVICE

Crear:

lib/services/saas-provisioning.service.ts

Responsabilidad:

provisionFromCapturedCheckout(checkoutSessionId, captureData)

Debe transformar una CheckoutSession confirmada en recursos SaaS.

No incluir UI.

No crear PayPal Order.

No calcular precios.

No aceptar datos del navegador.

---

## 20. FASE 2Q - TRANSACCIÓN ATÓMICA

Después de capture válido:

BEGIN TRANSACTION

1. Recargar CheckoutSession.
2. Bloquear/revalidar estado.
3. Comprobar que no está COMPLETED.
4. Comprobar captureId único.
5. Crear Payment.
6. Crear User si procede.
7. Crear Tenant.
8. Crear License.
9. Crear Subscription.
10. Crear Invoice.
11. Crear Membership OWNER.
12. Crear System Domain.
13. Guardar paymentId.
14. Guardar tenantId.
15. Guardar capturedAt.
16. Guardar completedAt.
17. Marcar CheckoutSession COMPLETED.
18. Guardar consumedAt.

COMMIT

Ante error crítico:

ROLLBACK

No devolver éxito completo con recursos parciales.

---

## 21. FASE 2R - NO FALSE SUCCESS

Eliminar en el flujo crítico patrones como:

```
try
create subscription
catch
warning
continue
```

y:

```
try
create invoice
catch
warning
continue
```

Subscription e Invoice son parte del resultado comercial definido por FenixCMS y deben tener una política explícita:

- o forman parte de la misma transacción;
- o el resultado se marca FAILED\_RETRYABLE y se recupera mediante reconciliación.

Nunca:

success = true

cuando una operación crítica no terminó.

---

## 22. FASE 2S - PAYMENT

Payment debe representar el pago real.

Debe contener como mínimo:

provider = PAYPAL

paymentType = SAAS\_LICENSE

providerTransactionId = captureId o identificador único correctamente diseñado

amount = amount capturado

currency = currency capturada

status = COMPLETED

paidAt = timestamp real

tenantId = Tenant creado

payment debe estar vinculado a CheckoutSession.

Una misma captura no puede producir dos Payments.

---

## 23. FASE 2T - TENANT

El Tenant final se crea después de la captura válida.

Los datos comerciales proceden del snapshot de CheckoutSession.

No utilizar datos arbitrarios del request de capture.

Debe existir la relación con:

Application

Plan

Owner

License

Subscription

Domain

---

## 24. FASE 2U - LICENSE

La licencia debe guardar:

tenantId

applicationId

planId

startsAt

expiresAt

price

currency

billingPeriod

activationLimit

licenseKeyHash

displayKey

La clave debe seguir utilizando la generación criptográfica existente.

No aceptar una licenseKey enviada por el cliente.

---

## 25. FASE 2V - SUBSCRIPTION

La suscripción inicial debe tener:

tenantId

planId

provider

billingPeriod

amount

currency

status  
currentPeriodStart  
currentPeriodEnd

No inventar providerSubscriptionId.

No marcar autoRenew si todavía no existe una integración real de renovación automática.

La renovación PayPal debe ser una fase funcional específica.

---

## 26. FASE 2W - INVOICE

La Invoice debe estar relacionada con:

Tenant  
Subscription  
Payment

Debe utilizar el importe real capturado y validado.

Debe quedar PAID únicamente cuando Payment esté realmente COMPLETED.

invoiceNumber debe ser unique.

---

## 27. FASE 2X - OWNER Y MEMBERSHIP

Si el usuario propietario no existe:

- crearlo;
- hash de password;
- crear Membership OWNER.

Si existe:

- comprobar reglas de pertenencia;
- no reasignar silenciosamente un usuario perteneciente a otro Tenant si la política no lo permite.

No utilizar contraseña fija.

---

## 28. FASE 2Y - DOMINIO DE SISTEMA

Crear:

slug.fenixcms.es

Este es el dominio inicial del Tenant.

Custom domain, si se solicita, debe permanecer:

PENDING\_VERIFICATION

No asignar verified=true sin comprobación real de DNS.

El DNS/SSL completo se tratará en fase posterior.

---

## 29. FASE 2Z - IDEMPOTENCIA

Casos que deben ser idempotentes:

- doble click;
- refresh en billing/success;
- retry del browser;
- retry del proveedor;
- webhook repetido;
- Browser + Webhook simultáneos.

Resultado:

1 CheckoutSession  
1 Payment

1 Tenant  
1 License  
1 Subscription  
1 Invoice  
1 Membership

y no más.

---

## 30. FASE 2AA - CONCURRENCIA

La posibilidad real:

Browser capture

+

Webhook

+

Retry

debe resolverse mediante PostgreSQL.

Usar:

- unique captureId;
- status PROVISIONING;
- transacción;
- comprobaciones condicionales;
- relectura posterior;
- constraints únicas.

No usar un Set en memoria como fuente de verdad.

---

## 31. FASE 2AB - PAYPAL WEBHOOK

Modificar:

app/api/webhooks/paypal/route.ts

y:

lib/services/webhook.service.ts

El flujo:

rawBody

-> verify signature

-> parse JSON

-> eventId

-> idempotency

-> persist event

-> process event

-> mark processed

PayPal:

<https://developer.paypal.com/api/rest/webhooks/rest/>

---

## 32. FASE 2AC - VERIFICACIÓN CRIPTOGRÁFICA WEBHOOK

No aceptar:

firma Base64 válida

como prueba suficiente.

No aceptar:

certUrl contiene paypal.com

como prueba suficiente.

Usar el endpoint oficial:

/v1/notifications/verify-webhook-signature

o la implementación criptográfica oficial equivalente.

PayPal:

<https://developer.paypal.com/api/invoicing/webhooks>

---

## 33. FASE 2AD - EVENTOS PAYPAL

CHECKOUT.ORDER.APPROVED

-> APPROVED/CAPTURE\_PENDING

-> no provisioning final.

PAYMENT.CAPTURE.PENDING

-> estado pendiente

-> no provisioning final.

PAYMENT.CAPTURE.COMPLETED

-> reconciliar y provisionar idempotentemente.

PAYMENT.CAPTURE.DENIED

-> FAILED/DENIED

-> no provisioning.

CHECKOUT.PAYMENT-APPROVAL.REVERSED

-> registrar reverso y reconciliar.

BILLING.SUBSCRIPTION.PAYMENT.SUCCEEDED

-> renovación, no compra inicial.

---

## 34. FASE 2AE - IDEMPOTENCIA DE WEBHOOK

WebhookEvent.eventId debe ser unique.

Mismo eventId:

Primer intento:

process.

Segundo intento:

no repetir side effects.

HTTP 200 para el duplicado ya procesado.

No depender del almacenamiento en memoria.

---

## 35. FASE 2AF - RECOVERY

Caso:

PayPal = COMPLETED

PostgreSQL provisioning = error temporal

No volver a cobrar.

Persistir la evidencia de capture.

Dejar CheckoutSession en estado recuperable:

PROVISIONING

o:

FAILED\_RETRYABLE

según la máquina de estados final.

Crear proceso:

reconcilePendingCheckoutSessions()

Este proceso debe poder terminar el provisioning de forma idempotente.

No ejecutarlo desde el browser.

---

## 36. FASE 2AG - AUTH SIN PASSWORDS MAESTRAS

Modificar:

lib/services/auth.service.ts

Eliminar:

Patricia1980@  
admin123  
fenix2026

Eliminar cualquier:

password === 'valor fijo'

Eliminar cuentas built-in como mecanismo de acceso de producción.

No introducir una nueva contraseña fija.

---

## 37. FASE 2AH - AUTH SOLO POSTGRESQL EN PRODUCCIÓN

En producción:

POST /api/auth/login

-> PostgreSQL User

-> PasswordService.verifyPassword

-> SessionService

-> secure httpOnly cookie

Si PostgreSQL falla:

ERROR

No:

inMemoryUserStore

no fallback login

no password maestra.

---

## 38. FASE 2AI - BOOTSTRAP DEL PRIMER SUPER ADMIN

Crear opcionalmente:

scripts/bootstrap-super-admin.ts

Usar variables de entorno:

FENIXCMS\_BOOTSTRAP\_ADMIN\_EMAIL

FENIXCMS\_BOOTSTRAP\_ADMIN\_PASSWORD

Reglas:

- solo si no existe SUPER\_ADMIN;
- hash antes de guardar;
- no imprimir password;
- no guardar password en Git;
- no usar bootstrap como login diario.

---

## 39. FASE 2AJ - SESSION SECURITY

Mantener:

httpOnly  
secure en production  
sameSite coherente  
path /  
maxAge controlado

Session debe seguir persistiendo en PostgreSQL.

---

## 40. FASE 2AK - STRIPE

No permitir que Stripe parezca activo si no existe Stripe Checkout real.

Eliminar URL ficticia del tipo:

checkout.stripe.com/pay/<sessionId>

si no procede de una verdadera Checkout Session creada por Stripe.

Mientras Stripe no esté terminado:

stripeEnabled = false

o rechazar el provider Stripe en producción.

No simular pagos.

---

## 41. FASE 2AL - SUPER ADMIN

Modificar:

lib/services/super-admin.service.ts

En producción no usar INITIAL\_\* como sustituto de PostgreSQL cuando una consulta falla.

Reemplazar patrones del tipo:

count().catch(() => INITIAL\_TENANTS.length)

por:

- error controlado;
- estado unavailable;
- 500/503 según endpoint.

No mostrar métricas inventadas.

---

## 42. FASE 2AM - STORECONTEXT

Modificar:

lib/storeContext.tsx

La función buyLicenseWithPayPal debe:

1. llamar checkout server-side;
2. recibir approvalUrl;
3. redirigir;
4. no fabricar Tenant;
5. no fabricar License;
6. no generar claves con Math.random;
7. actualizar estado local solo con respuesta confirmada del servidor.

El estado local nunca determina que una compra esté pagada.

---

## 43. FASE 2AN - PRIVACIDAD

No devolver innecesariamente al navegador:

- billingAddress;
- metadata interna;
- OAuth;
- Authorization headers;
- secrets;
- stack traces;
- datos de otros tenants.

No usar localStorage como prueba de pago.

---

## 44. FASE 2AO - ERRORES TIPADOS

Crear códigos de error consistentes:

CHECKOUT\_SESSION\_NOT\_FOUND  
CHECKOUT\_SESSION\_EXPIRED  
CHECKOUT\_SESSION\_ALREADY\_COMPLETED  
PAYPAL\_ORDER\_NOT\_FOUND  
PAYPAL\_ORDER\_MISMATCH  
PAYPAL\_CAPTURE\_NOT\_COMPLETED  
PAYPAL\_AMOUNT\_MISMATCH  
PAYPAL\_CURRENCY\_MISMATCH  
PAYPAL\_REFERENCE\_MISMATCH  
PAYPAL\_CAPTURE\_ALREADY\_USED  
PROVISIONING\_FAILED  
PROVISIONING\_RETRYABLE  
PAYMENT\_ALREADY\_COMPLETED

No devolver detalles internos al cliente.

---

## 45. FASE 2AP - HTTP STATUS

400

payload inválido.

401

autenticación requerida cuando proceda.

403

acceso prohibido.

404

CheckoutSession inexistente.

409

estado, importe, moneda o referencia incompatibles.

410

CheckoutSession expirada cuando se utilice este código.

500

error interno.

503

dependencia esencial no disponible.

---

## 46. FASE 2AQ - LOGGING

Registrar:

CHECKOUT\_SESSION\_CREATED  
PAYPAL\_ORDER\_CREATED  
PAYPAL\_APPROVAL\_RETURNED  
PAYPAL\_CAPTURE\_REQUESTED  
PAYPAL\_CAPTURE\_COMPLETED  
PAYPAL\_CAPTURE\_FAILED

SAAS\_PROVISIONING\_STARTED  
SAAS\_PROVISIONING\_COMPLETED  
SAAS\_PROVISIONING\_FAILED  
PAYPAL\_WEBHOOK\_VERIFIED  
PAYPAL\_WEBHOOK\_DUPLICATE  
CHECKOUT\_SESSION\_EXPIRED

No registrar:

- password;
- OAuth token;
- Client Secret;
- cookie;
- Authorization header;
- DATABASE\_URL.

---

## 47. FASE 2AR - CORRELATION ID

Utilizar CheckoutSession ID como identificador de correlación.

Debe permitir localizar:

CheckoutSession  
Payment  
Tenant  
License  
Subscription  
Invoice  
WebhookEvent  
AuditLog

---

## 48. FASE 2AS - PERSISTENCIA CRÍTICA

Eliminar como fuentes de verdad de producción:

FALLBACK\_SAAS\_PAYMENTS  
CONSUMED\_PAYPAL\_ORDERS  
MEMORY\_WEBHOOK\_EVENTS  
inMemoryUserStore

Pueden conservarse exclusivamente para tests aislados si se garantiza que no son alcanzables con NODE\_ENV=production.

---

## 49. FASE 2AT - TESTS FENIXCMS\_2

Crear:

tests/fenixcms-fase2.test.ts

### P01

Provision sin CheckoutSession:

REJECT.

### P02

Amount enviado por browser distinto:  
ignorado/rechazado.

### P03

Currency enviada por browser distinta:  
ignorada/rechazada.

### P04

Plan enviado durante capture distinto:  
ignorado/rechazado.

## **P05**

Application distinta durante capture:  
ignorada/rechazada.

## **P06**

TenantId arbitrario:  
ignorado/rechazado.

## **P07**

LicenseKey arbitraria:  
ignorada/rechazada.

## **P08**

APPROVED sin capture:  
no provisioning.

## **P09**

CAPTURE PENDING:  
no provisioning final.

## **P10**

CAPTURE DENIED:  
no provisioning.

## **P11**

CAPTURE COMPLETED:  
provisioning completo.

## **P12**

Amount mismatch:  
no provisioning.

## **P13**

Currency mismatch:  
no provisioning.

## **P14**

Reference mismatch:  
no provisioning.

## **P15**

Duplicate capture:  
idempotente.

## **P16**

Duplicate webhook:  
idempotente.

## **P17**

Webhook spoof:  
REJECT.

## **P18**

certUrl malicioso:  
REJECT.

## **P19**

timestamp inválido:  
REJECT.

## P20

CheckoutSession expirada:  
REJECT.

## P21

dos captures simultáneos:  
un provisioning.

## P22

browser + webhook simultáneos:  
un provisioning.

## P23

Subscription failure:  
no false success.

## P24

Invoice failure:  
no false success.

## P25

database unavailable + auth:  
no fallback login.

---

## 50. FASE 2AU - TEST DE CREDENCIALES

Crear una comprobación estática que detecte contraseñas hardcodeadas conocidas o patrones:

```
password ===  
admin123  
Patricia1980@  
fenix2026
```

El test debe fallar si aparecen en las rutas de producción.

---

## 51. FASE 2AV - TEST DE URLS FICTICIAS

Escanear producción para encontrar:

```
checkout.stripe.com/pay/  
paypal.com/checkoutnow?token=<sessionId generado internamente>
```

Una URL PayPal válida debe proceder del provider.

Una URL Stripe válida debe proceder de Stripe.

---

## 52. FASE 2AW - TEST DE FALLBACKS

Buscar:

```
INITIAL_PLANS  
INITIAL_APPLICATIONS  
INITIAL_TENANTS  
INITIAL_LICENSES  
tenant_demo  
FALLBACK_SAAS_PAYMENTS  
MEMORY_WEBHOOK_EVENTS  
inMemoryUserStore
```

Comprobar que no son alcanzables en production.

---

## 53. FASE 2AX - TEST MONEY

Casos:

0.01  
79.00  
79.01  
999999.99

El test debe demostrar comparación decimal consistente.

---

## 54. FASE 2AY - TEST PAYPAL SANDBOX

Debe existir una prueba de integración contra PayPal Sandbox que demuestre:

1. Create Order real.
2. Approval real.
3. Return.
4. Capture real.
5. COMPLETED.
6. Amount correcto.
7. Currency correcta.
8. Reference correcta.
9. Provisioning.
10. Idempotencia.

No llamar "test real" a un mock local.

---

## 55. FASE 2AZ - TEST E2E

Prueba de aceptación:

1. abrir landing SaaS;
2. seleccionar plan;
3. rellenar datos;
4. pulsar PayPal;
5. llegar a PayPal;
6. aprobar;
7. volver a FenixCMS;
8. capturar;
9. verificar estado;
10. comprobar Tenant;
11. comprobar License;
12. comprobar Subscription;
13. comprobar Invoice;
14. comprobar Membership;
15. comprobar acceso al CMS.

---

## 56. FASE 2BA - TEST RECOVERY

Simular:

PayPal capture COMPLETED

+

error temporal de DB durante provisioning.

Resultado esperado:

- evidencia de pago conservada;
- CheckoutSession recuperable;
- recovery posterior;
- no segundo cobro;
- no registros duplicados.

---

## 57. FASE 2BB - CONFIGURACIÓN LIVE

En producción:

```
NODE_ENV=production
APP_URL=https://dominio-real
DATABASE_URL=postgresql://...
PAYPAL_MODE=live
PAYPAL_BASE_URL=https://api-m.paypal.com
PAYPAL_CLIENT_ID=...
PAYPAL_CLIENT_SECRET=...
PAYPAL_WEBHOOK_ID=...
```

APP\_URL debe ser HTTPS.

No usar localhost.

No usar credenciales Sandbox en production.

Documentación PayPal:

<https://developer.paypal.com/api/rest/integration/orders-api>

---

## 58. FASE 2BC - WEBHOOK URL LIVE

La URL debe ser accesible desde Internet:

<https://dominio-real/api/webhooks/paypal>

Configurar en PayPal el webhook correspondiente.

No utilizar endpoint local.

---

## 59. FASE 2BD - SEPARACIÓN MOCK / SANDBOX / LIVE

Todo informe debe identificar claramente:

MOCK:

respuesta local simulada.

SANDBOX:

API real de PayPal Sandbox.

LIVE:

API real de PayPal Live.

Un PASS de MOCK no autoriza producción.

---

## 60. FASE 2BE - ARCHIVOS PRINCIPALES A MODIFICAR

Como mínimo revisar:

prisma/schema.prisma

app/api/billing/checkout/route.ts

app/api/billing/capture/route.ts

app/api/billing/verify/route.ts

app/api/tenants/provision/route.ts

app/api/webhooks/paypal/route.ts

components/saas/SaaS Landing.tsx

lib/services/saas-checkout.service.ts

lib/services/paypal-gateway.service.ts

lib/services/payment.service.ts

lib/services/tenant.service.ts  
lib/services/license.service.ts  
lib/services/subscription.service.ts  
lib/services/invoice.service.ts  
lib/services/webhook.service.ts  
lib/services/plan.service.ts  
lib/services/auth.service.ts  
lib/auth/session.ts  
lib/auth/password.ts  
lib/storeContext.tsx  
lib/services/super-admin.service.ts  
scripts/run-all-tests.ts

Crear nuevos servicios/rutas cuando sean necesarios.

---

## 61. FASE 2BF - ORDEN EXACTO DE IMPLEMENTACIÓN

FASE 2A  
CheckoutSession.

FASE 2B  
Prisma migration.

FASE 2C  
Application + Plan source of truth.

FASE 2D  
Commercial snapshot.

FASE 2E  
Checkout validation.

FASE 2F  
PayPal Create Order.

FASE 2G  
Redirect real.

FASE 2H  
Billing success/cancel.

FASE 2I  
Capture API.

FASE 2J  
Exact capture validation.

FASE 2K  
SaaS Provisioning Service.

FASE 2L  
Atomic transaction.

FASE 2M  
Concurrency/idempotency.

FASE 2N  
Webhooks.

FASE 2O  
Recovery.

FASE 2P  
Authentication cleanup.

FASE 2Q  
Super Admin fallbacks.

FASE 2R  
Stripe disabled until real.

FASE 2S  
Tests.

FASE 2T  
Sandbox E2E.

FASE 2U  
Lint/build.

---

## 62. PROMPT MAESTRO PARA AI STUDIO

### INICIO FASE FENIXCMS\_2

DONDE SE EJECUTA:  
AI STUDIO / repositorio fenixcms-sl-spec/moreno / rama main

REGLA ABSOLUTA:  
NO PAYMENT CAPTURED = NO PROVISIONING

OBJETIVO:  
Completar el flujo comercial FenixCMS con CheckoutSession persistente, PayPal approval real, capture server-side, validación exacta de amount/currency/reference, provisioning atómico, idempotencia, recovery, webhooks verificados y eliminación de credenciales maestras.

ANTES DE MODIFICAR:

1. Audita todo el código relacionado.
2. Lista los archivos afectados.
3. Describe el flujo actual.
4. Enumera los problemas reales.
5. No marques PASS por un mock.

CHECKOUTSESSION:  
Crear entidad de plataforma no tenant-scoped.  
Guardar snapshot comercial.  
Añadir unique constraints e índices.  
Crear migración nueva.

CATÁLOGO:  
Application y Plan deben provenir de PostgreSQL.  
Plan debe pertenecer a Application.  
Plan debe estar activo.  
No fallback a primera Application o primer Plan.

CREATE CHECKOUT:  
Validar request con Zod.  
No aceptar amount/currency/tenantId/license/subscription como autoridad.  
Crear CheckoutSession.  
Crear PayPal Order real.  
Guardar paypalOrderId.  
Devolver approvalUrl real.

FRONTEND:  
SaasLanding debe redirigir al approvalUrl.  
No debe llamar capture inmediatamente.  
No mostrar éxito antes de capture.

SUCCESS:  
El retorno de PayPal no significa pago completado.  
Usar el retorno únicamente para iniciar verificación/capture.

CAPTURE:  
Aceptar paypalOrderId.  
Buscar CheckoutSession.  
Comprobar expiry/state.  
Capturar en PayPal server-side.  
Comprobar:  
- order id exacto;  
- capture id real;

- status COMPLETED;
- amount exacto;
- currency exacta;
- custom/reference exacta.

Si falla una comprobación: NO PROVISIONING.

#### PROVISIONING:

Crear SaaSProvisioningService.

Después del capture válido, ejecutar una transacción PostgreSQL que incluya:

Payment

User

Tenant

License

Subscription

Invoice

Membership OWNER

System Domain

CheckoutSession COMPLETED

No devolver success si falla un recurso crítico.

#### IDEMPOTENCIA:

Browser + webhook + retry no deben duplicar recursos.

Usar unique constraints y estado PROVISIONING.

No usar Sets en memoria como autoridad.

#### WEBHOOK:

Verificación criptográfica oficial.

EventId unique.

APPROVED no provisiona.

CAPTURE.COMPLETED reconcilia.

DENIED/PENDING no provisionan.

#### RECOVERY:

Si PayPal está COMPLETED pero provisioning falla temporalmente:

conservar evidencia del pago;

dejar estado recuperable;

reintentar sin volver a cobrar.

#### AUTH:

Eliminar Patricia1980@, admin123, fenix2026.

Eliminar password comparisons hardcoded.

Eliminar fallback auth en production.

Production auth = PostgreSQL only.

Crear bootstrap seguro mediante variables de entorno y hash.

#### STRIPE:

Si no existe Stripe Checkout real:

desactivar Stripe en production.

No devolver URLs ficticias.

#### SUPER ADMIN:

No usar INITIAL\_\* como fuente de verdad en production.

No inventar métricas.

#### TESTS:

Crear/actualizar tests negativos y positivos:

- direct provisioning;
- amount tampering;
- currency tampering;
- plan tampering;
- application tampering;
- tenant tampering;
- license tampering;
- APPROVED;
- PENDING;
- DENIED;
- COMPLETED;
- mismatch;
- duplicate capture;
- concurrent capture;

- duplicate webhook;
- forged webhook;
- expired session;
- subscription failure;
- invoice failure;
- DB failure;
- master password detection;
- fake URL detection.

VALIDACIÓN FINAL:

Ejecutar:

```
npm ci
npx prisma generate
npm run lint
npm test
npm run build
```

Además ejecutar integración PayPal Sandbox.

INFORME FINAL:

1. archivos creados;
2. archivos modificados;
3. migración;
4. endpoints;
5. estados;
6. idempotencia;
7. recovery;
8. tests;
9. lint;
10. build;
11. variables de entorno;
12. blockers restantes.

No declares FENIXCMS\_2 terminada si queda cualquiera de los bloqueadores críticos.

**FIN FASE FENIXCMS\_2**

---

## 63. CRITERIOS DE ACEPTACIÓN FINAL

FENIXCMS\_2 solo queda terminada cuando:

- ☐ CheckoutSession persistente.
- ☐ CheckoutSession no depende de Tenant existente.
- ☐ Application validada.
- ☐ Plan validado.
- ☐ Plan pertenece a Application.
- ☐ Precio server-side.
- ☐ Moneda server-side.
- ☐ PayPal Order real.
- ☐ Approval URL real.
- ☐ Redirect real a PayPal.
- ☐ Return seguro.
- ☐ Capture server-side.
- ☐ COMPLETED obligatorio.
- ☐ Amount exacto.
- ☐ Currency exacta.
- ☐ Reference exacta.
- ☐ Capture único.

- ☐ Payment único.
- ☐ Tenant único.
- ☐ License única.
- ☐ Subscription única.
- ☐ Invoice única.
- ☐ Membership única.
- ☐ Webhook criptográficamente verificado.
- ☐ Webhook idempotente.
- ☐ Recovery sin segundo cobro.
- ☐ Sin passwords maestras.
- ☐ Sin auth fallback en production.
- ☐ Stripe no simulado.
- ☐ Super Admin sin métricas demo en production.
- ☐ Tests positivos y negativos.
- ☐ PayPal Sandbox E2E.
- ☐ npm run lint PASS.
- ☐ npm test PASS.
- ☐ npm run build PASS.

---

## 64. DEFINICIÓN DE TERMINADO

FENIXCMS\_2 no se considera terminada solo porque compile.

La evidencia mínima debe demostrar:

```
CREATE ORDER REAL
+
APPROVAL REAL
+
CAPTURE REAL
+
EXACT MATCH
+
ATOMIC PROVISIONING
+
WEBHOOK VERIFICADO
+
IDEMPOTENCIA
+
RECOVERY
+
AUTH SIN MASTER PASSWORD
+
POSTGRESQL COMO FUENTE DE VERDAD
+
TESTS
+
LINT
+
BUILD
+
PAYPAL SANDBOX E2E
```

Solo después de esta evidencia debe comenzarse la siguiente auditoría para decidir el despliegue definitivo al VPS.

---

## 65. REFERENCIAS TÉCNICAS

PayPal Orders API:  
<https://developer.paypal.com/api/rest/integration/orders-api>

PayPal Checkout Standard:  
<https://developer.paypal.com/platforms/checkout/standard/integrate>

PayPal Create Order:  
<https://developer.paypal.com/sdk/orders/v2/orders-create/>

PayPal Webhooks:  
<https://developer.paypal.com/api/rest/webhooks/rest/>

PayPal Webhook Verification:  
<https://developer.paypal.com/api/invoicing/webhooks>

Estas referencias deben utilizarse para comprobar el flujo vigente de PayPal cuando AI Studio implemente la fase.

---

## 66. NOTA DE ALCANCE

FENIXCMS\_2 es exclusivamente la fase de cierre del checkout/licenciamiento/provisioning.

Quedan fuera de esta fase:

- Storage real completo.
- eliminación completa de media demo;
- DNS/SSL automático completo;
- rate limiting distribuido;
- migración completa de Float a Decimal;
- CMS editor completo;
- SEO completo;
- Marketplace completo;
- plugins/temas como sandbox real;
- Stripe real completo;
- observabilidad avanzada.

Estos puntos se revisarán después de que el flujo de venta de licencia quede cerrado correctamente.

FIN DEL DOCUMENTO FENIXCMS\_2

**FIN FASE FENIXCMS\_2**